

ЕГЭ Информатика — Задание 13

Сети и IP-адресация

Теория

Задание №13 проверяет понимание **основ сетей и IP-адресации**. Обычно это базовое задание, на которое уходит около $\approx$ **3 минут**, а при хорошем знании теории — **10–30 секунд**.

Что нужно знать:

1. IP-адрес
2. маска сети
3. сеть и её компоненты

IP-адрес

IP-адрес — уникальный идентификатор устройства в сети. Он состоит из 4 байтов, то есть из 32 бит. Каждый байт принимает значение от 0 до 255.

$$1 \text{ байт} = 8 \text{ бит}, \quad 4 \text{ байта} = 32 \text{ бита}$$

Пример IP-адреса: 192.168.42.17. В двоичной записи этот адрес выглядит так: 11000000.10101000.00101010.00000101

Алгоритм перевода в двоичную систему:

1. каждый байт переводится отдельно
2. если битов меньше 8, добавляются ведущие нули
3. байты записываются через точку в исходном порядке

Маска сети

Маска сети определяет, какая часть IP-адреса относится к сети, а какая — к узлу. Она имеет ту же структуру: 4 байта, то есть 32 бита.

Главное свойство маски: в двоичной записи сначала идут единицы, потом нули. Например,

11111111111111110000000000000000

корректно, а

111100010000

некорректно, потому что после нуля снова появляется единица.

Примеры корректных масок:

- 255.255.255.0
- 255.255.224.0
- 255.255.255.255
- 0.0.0.0

Некорректный пример: 11111111.11111111.00111101.00000000.

CIDR-запись

Маску можно записывать кратко: $/n$, где n — количество единиц в маске.

Например, маска 255.255.255.0 в двоичном виде равна 11111111.11111111.11111111.00000000, значит, в ней 24 единицы, то есть это $/24$.

Компоненты сети

Любая сеть содержит:

1. маску
2. адрес сети
3. широковещательный адрес
4. узлы

Адрес сети — первый IP-адрес сети. **Broadcast** — последний IP-адрес сети. **Узлы** — все остальные устройства сети.

$$\text{адрес сети} \neq \text{узел}, \quad \text{broadcast} \neq \text{узел}$$

Поэтому

$$\text{количество узлов} = \text{количество IP} - 2$$

Количество адресов и узлов

Если n — количество единиц в маске, то число нулей:

$$k = 32 - n$$

Тогда:

$$\text{количество IP-адресов} = 2^k, \quad \text{количество узлов} = 2^k - 2$$

Пример: маска 255.255.240.0 имеет вид 11111111.11111111.11110000.00000000. В ней 20 единиц, значит,

$$k = 32 - 20 = 12, \quad 2^{12} = 4096, \quad 4096 - 2 = 4094$$

Главное правило сети:

$$\text{первый адрес} = \text{адрес сети}, \quad \text{последний адрес} = \text{broadcast}$$

Все остальные адреса — узлы.

Python для задания 13

Ручной перевод IP и побитовые операции занимают слишком много времени, поэтому обычно используют модуль `ipaddress`.

```
from ipaddress import *  
net = ip_network("IP/MASK", False)
```

Полезные свойства:

```
net.network_address
net.broadcast_address
net.netmask
```

Если маска неизвестна, можно перебирать все варианты:

```
for m in range(33):
    net = ip_network(f"148.195.140.28/{m}", False)
    print(net)
```

Решение прототипов

1. Найти адрес сети

Условие. По заданным IP-адресу узла сети и маске определите адрес сети: IP-адрес: 10.8.248.131, маска: 255.255.224.0.

Решение. Адрес сети получается побитовым И между IP-адресом и маской. Удобнее всего решать через Python:

```
from ipaddress import *

net = ip_network(f"10.8.248.131/255.255.224.0", False)
print(net)
print(net[0])
print(net.network_address)
```

Что делает этот код:

- `ip_network(..., False)` создаёт объект сети по IP-адресу и маске
- параметр `False` разрешает передавать не адрес сети, а адрес любого узла из этой сети
- `print(net)` печатает всю сеть в сокращённом виде: 10.8.224.0/19
- `print(net[0])` печатает первый адрес сети, а первый адрес всегда является адресом сети
- `print(net.network_address)` обращается к специальному свойству объекта сети и тоже даёт адрес сети

То есть две последние строки выводят одно и то же значение. В этой задаче:

$$net[0] = net.network_address = 10.8.224.0$$

Получаем 10.8.224.0.

Ответ: 10.8.224.0.

2. Найти байт маски по IP и адресу сети

Условие. Для узла с IP-адресом 220.128.112.142 адрес сети равен 220.128.96.0. Чему равен третий слева байт маски? Ответ запишите в виде десятичного числа.

Решение. Эту задачу удобно решать перебором всех возможных масок и смотреть, при какой маске получится нужный адрес сети.

```
from ipaddress import *

for m in range(33):
    net = ip_network(f"220.128.112.142/{m}", False)
    print(net, net.netmask)
```

Как это работает:

- `m` — это количество единиц в маске, то есть префикс от /0 до /32
- для каждого значения строится сеть, в которую попадает адрес 220.128.112.142
- `print(net, net.netmask)` печатает сразу адрес сети и полную маску
- дальше нужно просто найти глазами строку, где адрес сети равен 220.128.96.0

В выводе появится строка вида:

```
220.128.96.0/19 255.255.224.0
```

Значит, нужная маска — 255.255.224.0. Теперь берём третий слева байт этой маски:

$$255.255.224.0 \Rightarrow 224$$

То есть здесь даже не обязательно вручную переводить байты в двоичную систему: достаточно перебрать маски, найти нужный адрес сети и считать ответ из найденной маски.

Ответ: 224.

3. Найти минимальное количество единиц в маске

Условие. Для узла с IP-адресом 148.195.140.28 адрес сети равен 148.195.140.0. Найдите наименьшее возможное количество единиц в двоичной записи маски подсети.

Решение. Эту задачу тоже удобно решать перебором всех возможных масок.

```
from ipaddress import *

for m in range(33):
    net = ip_network(f"148.195.140.28/{m}", False)
    print(net)
```

Как рассуждать по выводу:

- программа перебирает все префиксы от /0 до /32
- для каждого префикса она печатает адрес сети
- нужно найти все строки, где сеть равна 148.195.140.0
- из них выбрать **минимальное** значение `m`, потому что в задаче спрашивают минимальное количество единиц в маске

В выводе впервые встретится строка:

```
148.195.140.0/22
```

Это значит, что маска /22 уже подходит. Все более длинные маски тоже могут давать тот же адрес сети, но нам нужна именно минимальная.

То есть здесь не обязательно вручную переводить байты в двоичную систему: достаточно перебрать маски и взять первое подходящее значение префикса.

Ответ: 22.

4. Два узла в одной сети: найти максимальный байт маски

Условие. Два узла, находящиеся в одной сети, имеют IP-адреса 112.117.107.70 и 112.117.121.80. Укажите наибольшее возможное значение третьего слева байта маски сети. Ответ запишите в виде десятичного числа.

Решение. Это удобно решать перебором всех масок. Идея такая: для каждой маски строим сеть для первого и второго IP, потом проверяем три условия:

- первый IP действительно является узлом, а не адресом сети и не broadcast
- второй IP тоже является узлом
- обе сети совпадают

```
from ipaddress import *

ip1 = ip_address('112.117.107.70')
ip2 = ip_address('112.117.121.80')

for m in range(33):
    net1 = ip_network(f'{ip1}/{m}', False)
    net2 = ip_network(f'{ip2}/{m}', False)
    if net1[0] < ip1 < net1[-1]:
        if net2[0] < ip2 < net2[-1]:
            if net1 == net2:
                print(net1.netmask)
```

Что проверяет этот код:

- `net1[0] < ip1 < net1[-1]` означает, что `ip1` строго между первым и последним адресом сети
- значит, `ip1` точно не является ни адресом сети, ни широковещательным адресом
- аналогичная проверка делается для `ip2`
- `net1 == net2` означает, что оба адреса попали в одну и ту же сеть
- программа печатает все подходящие маски

Из напечатанных масок нужно взять ту, у которой третий байт максимален. Последняя подходящая маска будет:

255.255.224.0

Берём третий слева байт:

$$255.255.224.0 \Rightarrow 224$$

То есть дополнительная проверка правильная: мы заранее убеждаемся, что оба IP — это именно узлы сети, а не её границы.

Ответ: 224.

5. Найти количество узлов по маске

Условие. Для некоторой подсети используется маска 255.255.255.128. Сколько различных адресов компьютеров теоретически допускает эта маска, если два адреса (адрес сети и широковещательный) не используют?

Решение. Это можно посчитать формулой, а можно совсем быстро через Python:

```
from ipaddress import *

net = ip_network(f'0.0.0.0/255.255.255.128', False)
print(net.num_addresses - 2)
```

Почему это работает:

- здесь важна только маска, а конкретный адрес можно взять любой удобный, например 0.0.0.0
- объект `net` создаётся по этой маске
- `net.num_addresses` — это общее количество адресов в такой сети
- из этого количества вычитаем 2, потому что нельзя использовать адрес сети и broadcast

Для маски 255.255.255.128 программа получает:

$$128 - 2 = 126$$

Если решать вручную, то 255.255.255.128 — это маска /25, значит, остаётся 7 бит под узлы, всего адресов

$$2^7 = 128$$

и тогда

$$128 - 2 = 126$$

Ответ: 126.

6. Сколько существует подходящих масок

Условие. Для узла с IP-адресом 175.122.80.13 адрес подсети равен 175.122.80.0. Сколько существует различных возможных значений маски, если известно, что в этой сети не менее 60 узлов? Ответ запишите в виде десятичного числа.

Решение. Это удобно решать перебором всех масок с дополнительной проверкой на количество узлов:

```
from ipaddress import *

for m in range(33):
    net = ip_network(f'175.122.80.13/{m}', False)
    if net.num_addresses - 2 >= 60:
        print(net)
```

Как рассуждать по этому коду:

- перебираются все возможные префиксы от /0 до /32
- для каждого префикса строится сеть, в которую попадает адрес 175.122.80.13
- условие `net.num_addresses - 2 >= 60` оставляет только те сети, где не меньше 60 узлов
- дальше из вывода нужно взять только те строки, где адрес сети равен 175.122.80.0

В выводе останутся такие подходящие сети:

```
175.122.80.0/20
175.122.80.0/21
175.122.80.0/22
175.122.80.0/23
```

175.122.80.0/24
175.122.80.0/25
175.122.80.0/26

Их 7 штук, значит, существует 7 подходящих масок.

Здесь удобно именно считать количество подходящих строк в выводе.

Ответ: 7.

7. Найти номер компьютера в сети

Условие. Если маска подсети 255.255.252.0 и IP-адрес компьютера в сети 156.132.15.138, то чему будет равен номер компьютера в сети?

Решение. Это можно находить через смещение от адреса сети, но можно и просто перебрать все адреса сети по порядку:

```
from ipaddress import *

net = ip_network('156.132.15.138/255.255.252.0', False)

n = 0
for x in net:
    print(n, x)
    n += 1
```

Что делает этот код:

- создаётся сеть, в которую входит IP-адрес 156.132.15.138
- переменная `n` — это номер адреса внутри сети
- цикл `for x in net` перебирает все адреса сети подряд: от адреса сети до broadcast
- команда `print(n, x)` печатает номер и сам IP-адрес

Дальше нужно просто найти в выводе строку, где будет адрес 156.132.15.138. Для него программа напечатает:

```
906 156.132.15.138
```

Значит, номер этого компьютера в сети равен 906.

Почему это работает: перебор начинается с адреса сети, у него номер 0, затем номер увеличивается на 1 для каждого следующего адреса. Поэтому когда цикл доходит до нужного IP, число `n` и есть его номер в этой сети.

Ответ: 906.

8. Найти наибольший IP-адрес компьютера в сети

Условие. Сеть задана IP-адресом одного из входящих в неё узлов 218.194.82.148 и сетевой маской 255.255.255.192. Найдите наибольший IP-адрес в данной сети, который может быть назначен компьютеру. В ответе укажите найденный IP-адрес без разделителей.

Решение. Это удобно сделать совсем коротко через объект сети:

```
from ipaddress import *

net = ip_network('218.194.82.148/255.255.255.192', False)
print(net[-2])
```

Что делает этот код:

- создаётся сеть, в которую входит адрес 218.194.82.148
- объект `net` хранит все адреса сети по порядку
- `net[-1]` — это последний адрес сети, то есть broadcast
- `net[-2]` — это адрес перед broadcast, а значит, максимальный адрес, который можно выдать компьютеру

То есть логика здесь очень простая:

последний адрес = broadcast, предпоследний адрес = последний допустимый узел

Программа выводит:

218.194.82.190

По условию ответ нужно записать без точек, поэтому получаем:

21819482190

Ответ: 21819482190.

9. Сколько адресов удовлетворяют условию по двоичной записи

Условие. Сеть задана IP-адресом 214.96.0.0 и маской сети 255.240.0.0. Сколько в этой сети IP-адресов, у которых количество нулей в двоичной записи IP-адреса кратно трём?

Решение. Маска 255.240.0.0 соответствует префиксу /12. Значит, первые 12 бит фиксированы, остальные 20 бит перебираются.

Такие задачи удобнее всего решать программно: перебрать все адреса сети, перевести адрес в 32-битную строку и посчитать число нулей.

```
from ipaddress import *
net = ip_network("214.96.0.0/255.240.0.0")
count = 0
for ip in net:
    b = bin(int(ip))[2:].zfill(32)
    if b.count('0') % 3 == 0:
        count += 1
print(count)
```

Программа даёт значение 349526.

Ответ: 349526.

10. Сумма единиц в двоичной записи четвёртого байта

Условие. Сеть задана IP-адресом 123.222.111.192 и маской сети 255.255.255.248. Сколько в этой сети IP-адресов, для которых сумма единиц в двоичной записи четвёртого байта IP-адреса не делится без остатка на 3? В ответе укажите только число.

Решение. Эту задачу удобно решить полным перебором всех адресов сети:

```
from ipaddress import *

net = ip_network(f'123.222.111.192/255.255.255.248', 0)
k = 0
for x in net:
    # ip = bin(int(x))[2:].zfill(32)
    ip = f'{int(x):032b}'
    if ip[-8:].count('1') % 3 != 0:
        k += 1
print(k)
```

Что делает этот код:

- создаётся сеть по заданному IP и маске
- переменная `k` считает количество подходящих адресов
- цикл перебирает все IP-адреса этой сети
- `int(x)` переводит IP-адрес в целое число
- `f'{int(x):032b}'` записывает это число как двоичную строку длиной ровно 32 бита
- `ip[-8:]` берёт последние 8 бит, то есть четвёртый байт IP-адреса
- `count('1')` считает, сколько единиц в этом байте
- если это количество не делится на 3, увеличиваем счётчик

Здесь сеть имеет маску /29, значит, в ней всего 8 адресов. Программа просто проверяет каждый из них и считает подходящие.

В результате получаем:

5

Ответ: 5.